Análisis espacial — ATUS ZMM

Dónde ocurren los accidentes de tránsito de la Zona Metropolitana de Monterrey, 2019-2024, y si ese *dónde* tiene estructura: concentración, autocorrelación espacial global y local, puntos calientes, sensibilidad a la escala y persistencia en el tiempo.

Fuente: `data/processed/atus_zmm_limpio_geo.parquet` (379,294 registros × 71 columnas, con geometría de puntos). Ese archivo **no está versionado** — se genera con:

```
uv run limpiar-atus --geo
```

Las columnas en MAYÚSCULAS son del INEGI; las minúsculas las construye `geostats.limpieza`. Ver `docs/limpieza.md` y la selección de variables.

Qué hay aquí

§	Tema
1	Proyección y unidad de análisis: por qué hexágonos
2	Concentración espacial: Lorenz y Gini
3	Matriz de pesos espaciales
4	Autocorrelación global: I de Moran
5	Sensibilidad a la escala (MAUP)
6	Autocorrelación local: LISA
7	Puntos calientes: Getis-Ord G*
8	Persistencia: ¿el mapa de 2019 predice el de 2024?
9	La escala fina: intersecciones

> **Advertencia que atraviesa todo el documento.** Cada estadístico local se calcula sobre miles de celdas a la vez. Al 5 % de significancia, 2,241 pruebas producen **unos 112 falsos positivos por puro azar**. Todo mapa de clusters o puntos calientes de este notebook lleva corrección por tasa de falsos descubrimientos (Benjamini-Hochberg); se reporta cuántas celdas sobreviven a la corrección y cuántas no.

```
import warnings
warnings.filterwarnings("ignore")
```

```
import numpy as np
import pandas as pd
import geopandas as gpd
import matplotlib.pyplot as plt
from matplotlib.colors import LinearSegmentedColormap
from matplotlib.patches import Patch
from scipy import stats
from libpysal import weights
from esda.moran import Moran, Moran_Local
from esda.getisord import G_Local
from statsmodels.stats.multitest import multipletests

from geostats import espacial, rutas
from geostats.tabla_cruces import nombres_de_cruce

np.random.seed(42)

g = gpd.read_parquet(rutas.ATUS_ZMM_LIMPIO_GEO)
N = len(g)
print(f"{N:,} registros | CRS de origen: {g.crs.to_epsg()} | "
      f"{g.fecha.min():%Y-%m-%d} a {g.fecha.max():%Y-%m-%d}")
```

```
379,294 registros | CRS de origen: 4326 | 2019-01-01 a 2024-12-31
```

Estilo de las gráficas

Misma identidad que los notebooks anteriores. Lo que este añade son los **mapas categóricos** de clusters y puntos calientes: alto-alto en Rojo profundo, bajo-bajo en Azul Prusia, los dos atípicos espaciales en tonos intermedios y lo no significativo en gris. Es el mismo par azul/rojo validado para la escala divergente, con la diferencia de que aquí las categorías nunca son contiguas en una serie: cada una es un parche en el mapa.

```
# --- Identidad GeoStats ------------------------------------------------
AZUL     = "#005991"   # Azul Prusia ajustado -> datos
ROJO     = "#8B2C1A"   # Rojo profundo -> énfasis, títulos
OXIDO    = "#B15E2E"   # Rojo óxido -> detalle cálido
GRAFITO  = "#2C2C2C"   # texto y estructura
GRIS     = "#F2F2F2"   # fondos, rejilla, no significativo
BLANCO   = "#FFFFFF"
AZUL_RAMPA = ["#005991", "#1b77b8", "#4195d9"]

SECUENCIAL = LinearSegmentedColormap.from_list("geostats_seq", [BLANCO, AZUL])
DIVERGENTE = LinearSegmentedColormap.from_list("geostats_div", [AZUL, GRIS, ROJO])

TITULAR = ["Montserrat", "Helvetica Neue", "Arial", "DejaVu Sans"]
TEXTO   = ["Cormorant Garamond", "EB Garamond", "Georgia", "DejaVu Serif"]
CIFRAS  = ["Roboto Mono", "Menlo", "DejaVu Sans Mono"]
```

```
plt.rcParams.update({
    "figure.dpi": 130, "figure.facecolor": BLANCO, "axes.facecolor": BLANCO,
    "savefig.facecolor": BLANCO, "font.family": "sans-serif",
    "font.sans-serif": TITULAR, "font.serif": TEXTO, "font.monospace": CIFRAS,
    "font.size": 9, "axes.edgecolor": GRIS, "axes.labelcolor": GRAFITO,
    "axes.labelsize": 9, "axes.titlecolor": ROJO, "axes.titlesize": 11.5,
    "axes.titleweight": "bold", "axes.titlelocation": "left", "axes.titlepad": 16,
    "axes.grid": True, "axes.axisbelow": True, "grid.color": GRIS,
    "grid.linewidth": 1.0, "xtick.color": GRAFITO, "ytick.color": GRAFITO,
    "xtick.labelsize": 8.5, "ytick.labelsize": 8.5, "legend.frameon": False,
    "legend.fontsize": 8.5, "lines.linewidth": 1.8,
})

def pulir(ax, ejes=("top", "right", "left")):
    for lado in ejes:
        ax.spines[lado].set_visible(False)
    ax.tick_params(length=0)
    for etiqueta in ax.get_xticklabels() + ax.get_yticklabels():
        etiqueta.set_fontfamily("monospace")
    return ax

def mapa(ax, titulo=None, escala_km=10):
    """Un mapa no lleva ejes ni rejilla; lleva proporción 1:1 y una barra de escala."""
    ax.set_aspect("equal")
    ax.grid(False)
    ax.set_xticks([]); ax.set_yticks([])
    for lado in ax.spines.values():
        lado.set_visible(False)
    if titulo:
        ax.set_title(titulo)
    x0, x1 = ax.get_xlim(); y0, y1 = ax.get_ylim()
    bx = x0 + 0.04 * (x1 - x0); by = y0 + 0.04 * (y1 - y0)
    ax.plot([bx, bx + escala_km * 1000], [by, by], color=GRAFITO, lw=2, solid_capstyle="b
    ax.text(bx, by + 0.012 * (y1 - y0), f"{escala_km} km", fontsize=7.5,
            color=GRAFITO, fontfamily="monospace")
    return ax
```

---

1. Proyección y unidad de análisis

Proyectar antes de medir

Las coordenadas del INEGI vienen en WGS 84, en grados. A la latitud de Monterrey un grado de longitud mide ~100 km y uno de latitud ~111 km: **una distancia en grados no es una distancia**. Todo lo que sigue —áreas, vecindades, rejillas— exige una proyección métrica. Se usa UTM zona 14 Norte (EPSG:32614), que cubre la franja −102° a −96° donde está toda la ZMM.

```
g = g.to_crs(espacial.UTM_ZMM)
xmin, ymin, xmax, ymax = g.total_bounds
print(f"Extensión de la ZMM: {(xmax - xmin) / 1e3:.0f} km este-oeste × "
      f"{(ymax - ymin) / 1e3:.0f} km norte-sur")
```

```
Extensión de la ZMM: 104 km este-oeste × 121 km norte-sur
```

Por qué hexágonos y no AGEBs ni puntos

Los accidentes se agregan a una **rejilla hexagonal regular de 500 m de lado** (0.65 km² por celda, comparable a un AGEB urbano). Tres razones:

1. **Área constante.** Con celdas iguales, el conteo *es* la densidad. No hace falta un denominador de superficie, y ese es el primer denominador que siempre falta.
2. **Sin sesgo de frontera.** Los límites de AGEB se trazan sobre avenidas, que es justo donde caen los accidentes; un choque en Constitución cae en la frontera entre dos polígonos y su asignación es arbitraria. Un hexágono no sabe dónde están las calles.
3. **Vecindad sin ambigüedad.** Todo hexágono tiene seis vecinos idénticos. En una malla cuadrada las vecindades por vértice y por arista difieren; en un polígono irregular el número de vecinos va de 1 a 20.

La contraparte es que el hexágono no trae covariables censales; eso se resuelve cuando toque unir con el marco geoestadístico, no aquí.

Se conservan solo las celdas **con al menos un accidente en los seis años**. Una celda sin un solo accidente en 379,294 registros no tiene red vial donde pueda ocurrir uno: es cerro, presa o terreno baldío, no una zona de riesgo cero. Incluirlas inflaría artificialmente los clusters bajo-bajo con territorio donde la pregunta ni siquiera aplica.

```
LADO = 500
H = espacial.rejilla_hexagonal(g.total_bounds, lado=LADO)
AREA_KM2 = H.geometry.iloc[0].area / 1e6

union = gpd.sjoin(g[["geometry", "ANIO"]], H, predicate="within", how="inner")
ANIOS = sorted(g.ANIO.unique())
panel = union.pivot_table(index="hex", columns="ANIO", values="geometry",
                          aggfunc="size", fill_value=0)
panel["n"] = panel[ANIOS].sum(axis=1)
H = H.set_index("hex").join(panel, how="inner").reset_index()

print(f"Rejilla: lado {LADO} m, {AREA_KM2:.2f} km² por celda")
print(f"Celdas con ≥ 1 accidente: {len(H):,} | accidentes asignados: "
      f"{H.n.sum():,} de {N:,} ({H.n.sum() / N:.3%})")
print(f"Accidentes por celda — mediana {H.n.median():.0f}, media {H.n.mean():.0f}, "
      f"p90 {H.n.quantile(.9):.0f}, máximo {H.n.max():,}")
```

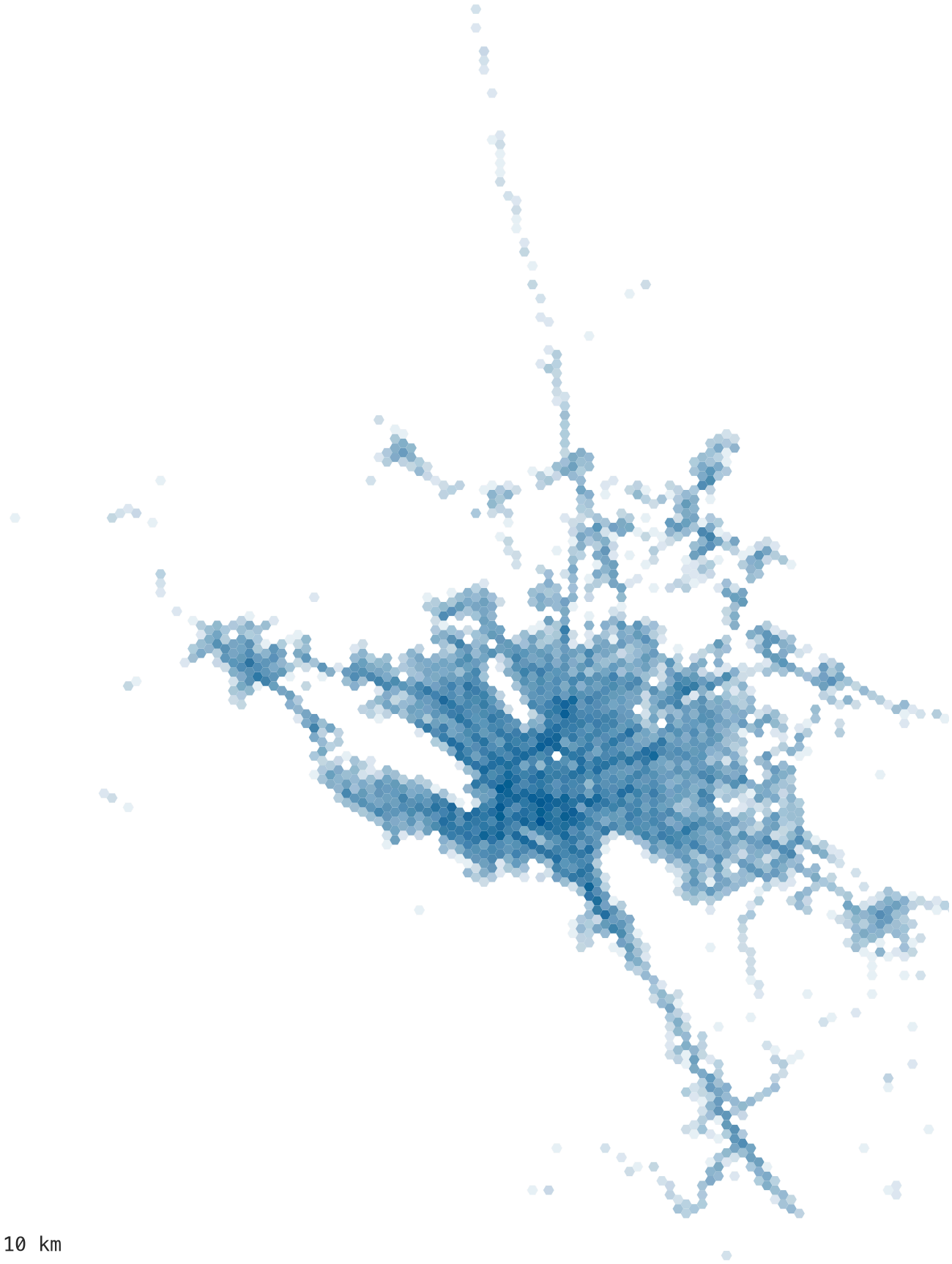
```
Rejilla: lado 500 m, 0.65 km² por celda
Celdas con ≥ 1 accidente: 2,344 | accidentes asignados: 379,293 de 379,294 (100.000%)
```

```
Accidentes por celda — mediana 30, media 162, p90 430, máximo 3,741
```

Un registro queda fuera por caer exactamente sobre una arista de la rejilla; es irrelevante. Lo que importa es la forma de la distribución: la mediana es 30 y el máximo 3,741. La celda típica y la celda extrema están **dos órdenes de magnitud** aparte.

```
fig, ax = plt.subplots(figsize=(8.5, 8))
H.plot(column=np.log10(H.n), cmap=SECUENCIAL, ax=ax, linewidth=0, legend=True,
       legend_kwds={"shrink": 0.5, "label": "log₁₀ accidentes", "pad": 0.01})
mapa(ax, "Dónde ocurren los accidentes de la ZMM")
plt.tight_layout(); plt.show()
```

Dónde ocurren los accidentes de la ZMM



Accidentes por celda hexagonal de 500 m, 2019-2024. Escala logarítmica: sin ella, el mapa es blanco con cinco puntos oscuros.

---

2. Concentración espacial

Antes de preguntar si las celdas vecinas se parecen, conviene medir cuánto se concentra el fenómeno. La curva de Lorenz ordena las celdas de menor a mayor y acumula: si los accidentes se repartieran por igual, sería la diagonal.

```
def lorenz(x):
    x = np.sort(np.asarray(x, dtype=float))
    acum = np.cumsum(x) / x.sum()
    return np.linspace(0, 1, len(x) + 1), np.insert(acum, 0, 0)

def gini(x):
    x = np.sort(np.asarray(x, dtype=float))
    n = len(x)
    return (2 * np.sum(np.arange(1, n + 1) * x) / (n * x.sum())) - (n + 1) / n

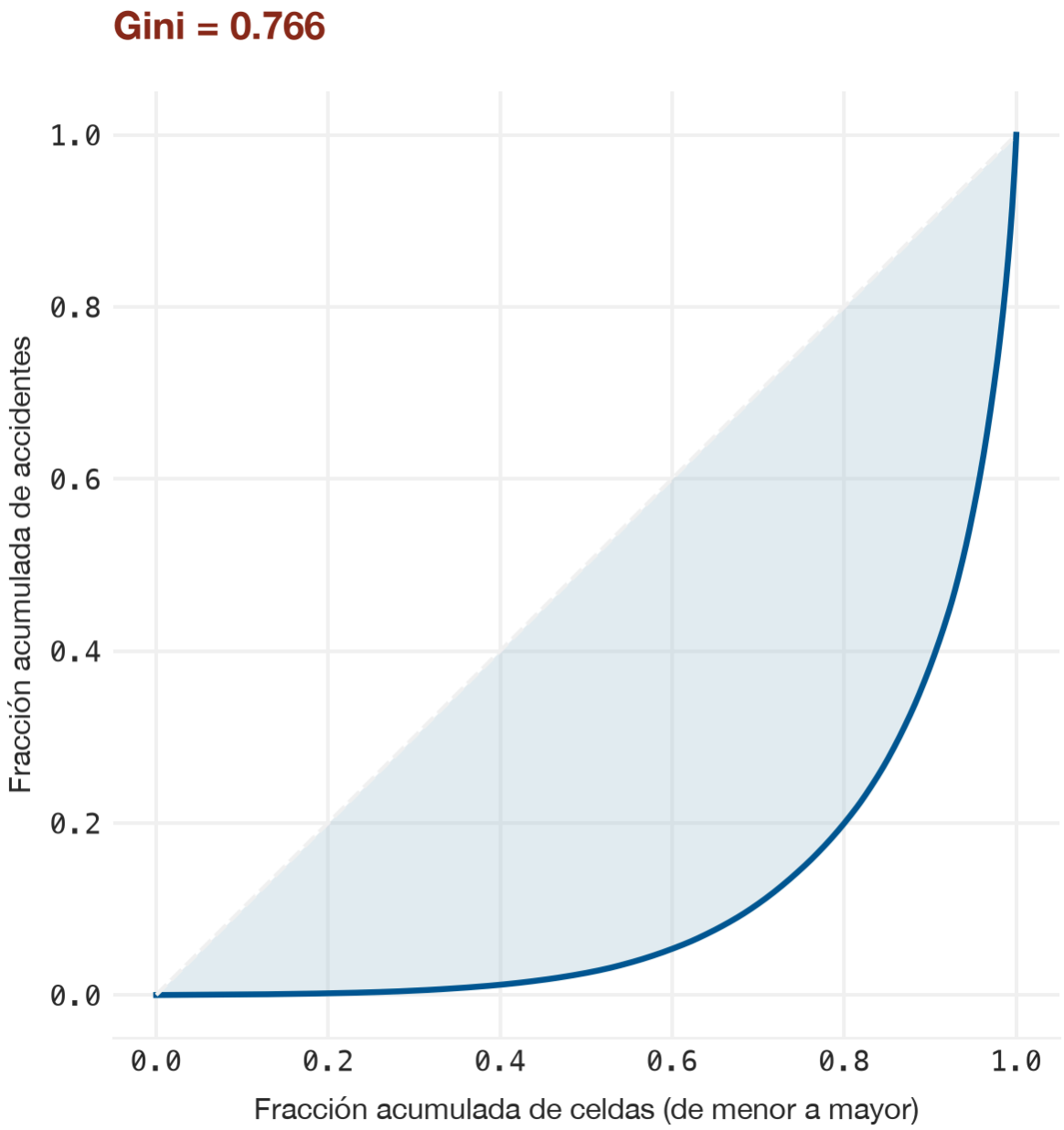
def concentracion(x, fracciones=(0.01, 0.05, 0.10, 0.20)):
    s = pd.Series(x).sort_values(ascending=False)
    return {f: s.head(max(1, int(len(s) * f))).sum() / s.sum() for f in fracciones}

G_HEX = gini(H.n)
conc = concentracion(H.n)
print(f"Gini espacial (hexágonos): {G_HEX:.3f}")
for f, v in conc.items():
    print(f"  el {f:>4.0%} de las celdas ({int(len(H) * f):>4,}) concentra el {v:.1%} de
```

```
Gini espacial (hexágonos): 0.766
  el   1% de las celdas (  23) concentra el 14.9% de los accidentes
  el   5% de las celdas ( 117) concentra el 43.7% de los accidentes
  el  10% de las celdas ( 234) concentra el 61.7% de los accidentes
  el  20% de las celdas ( 468) concentra el 80.0% de los accidentes
```

```
px, py = lorenz(H.n)
fig, ax = plt.subplots(figsize=(5.2, 5))
ax.fill_between(px, py, px, color=AZUL, alpha=0.10, linewidth=0)
ax.plot(px, py, color=AZUL)
ax.plot([0, 1], [0, 1], color=GRIS, lw=1.2, ls="--")
ax.set_xlabel("Fracción acumulada de celdas (de menor a mayor)")
ax.set_ylabel("Fracción acumulada de accidentes")
ax.set_title(f"Gini = {G_HEX:.3f}")
```

```
ax.set_aspect("equal")
pulir(ax); plt.tight_layout(); plt.show()
```



Curva de Lorenz de la distribución espacial. La diagonal sería reparto uniforme; el área entre ambas es el Gini.

Un Gini de 0.76 es el de una distribución de ingreso extremadamente desigual. **El 5 % de la superficie con red vial concentra más del 40 % de los accidentes.** Ese solo número justifica el enfoque territorial: intervenir un territorio pequeño toca una fracción grande del problema.

---

3. Matriz de pesos espaciales

Toda medida de autocorrelación espacial necesita que se defina *vecino*. Para la rejilla hexagonal la elección es única: **contigüidad**, dos celdas son vecinas si comparten arista. Cada hexágono interior tiene seis vecinos; en el borde de la mancha urbana menos.

La matriz se **estandariza por fila**: cada vecino de una celda pesa 1/(número de vecinos). Así el rezago espacial de una celda es el promedio de sus vecinas, y el estadístico no depende de cuántas tenga.

```
W0 = weights.Queen.from_dataframe(H, use_index=False, silence_warnings=True)
islas = H.iloc[W0.islands]
print(f"Celdas isla (ningún vecino con accidentes): {len(islas)} "
      f"con {islas.n.sum():,} accidentes ({islas.n.sum() / N:.2%}); máximo por isla {isla
```

```
Celdas isla (ningún vecino con accidentes): 103 con 256 accidentes (0.07%); máximo por
isla 28
```

Las islas son celdas cuyo entorno inmediato no tiene un solo accidente en seis años: choques aislados en caminos rurales o en los bordes del área metropolitana. No tienen con quién correlacionarse, y un estadístico local sobre ellas es una división entre cero. **Se retiran**; son el 0.07 % de los registros.

```
H = H.drop(index=W0.islands).reset_index(drop=True)
W = weights.Queen.from_dataframe(H, use_index=False, silence_warnings=True)
W.transform = "r"
print(f"Celdas en el análisis: {len(H):,} | vecinos por celda: media {W.mean_neighbors:.2
      f"mínimo {W.min_neighbors}, máximo {W.max_neighbors} | islas: {len(W.islands)}")
```

```
Celdas en el análisis: 2,241 | vecinos por celda: media 3.75, mínimo 1, máximo 6 | islas:
0
```

La variable que se analiza

Con mediana 30 y máximo 3,741, el conteo crudo tiene una cola que domina cualquier estadístico basado en productos cruzados: cinco celdas decidirían el resultado. Se trabaja sobre **log(1 + n)**, que comprime la cola sin reordenar. El I de Moran se calcula sobre ambas versiones para verificar que la conclusión no depende de la transformación.

```
y = np.log1p(H.n.values)
y_crudo = H.n.values.astype(float)
```

---

4. Autocorrelación global: I de Moran

La pregunta: ¿las celdas con muchos accidentes tienden a estar rodeadas de celdas con muchos accidentes? El I de Moran es la correlación entre el valor de cada celda y el promedio de sus vecinas. Bajo independencia espacial su esperanza es $-1/(n-1) \approx 0$; positivo significa que lo parecido se junta.

La inferencia es por **permutaciones**: se barajan los valores entre celdas 999 veces y se mira dónde cae el I observado respecto a esa distribución. No supone normalidad, que aquí no se cumple.

```
PERM = 999
moran = Moran(y, W, permutations=PERM)
```

```
moran_crudo = Moran(y_crudo, W, permutations=PERM)

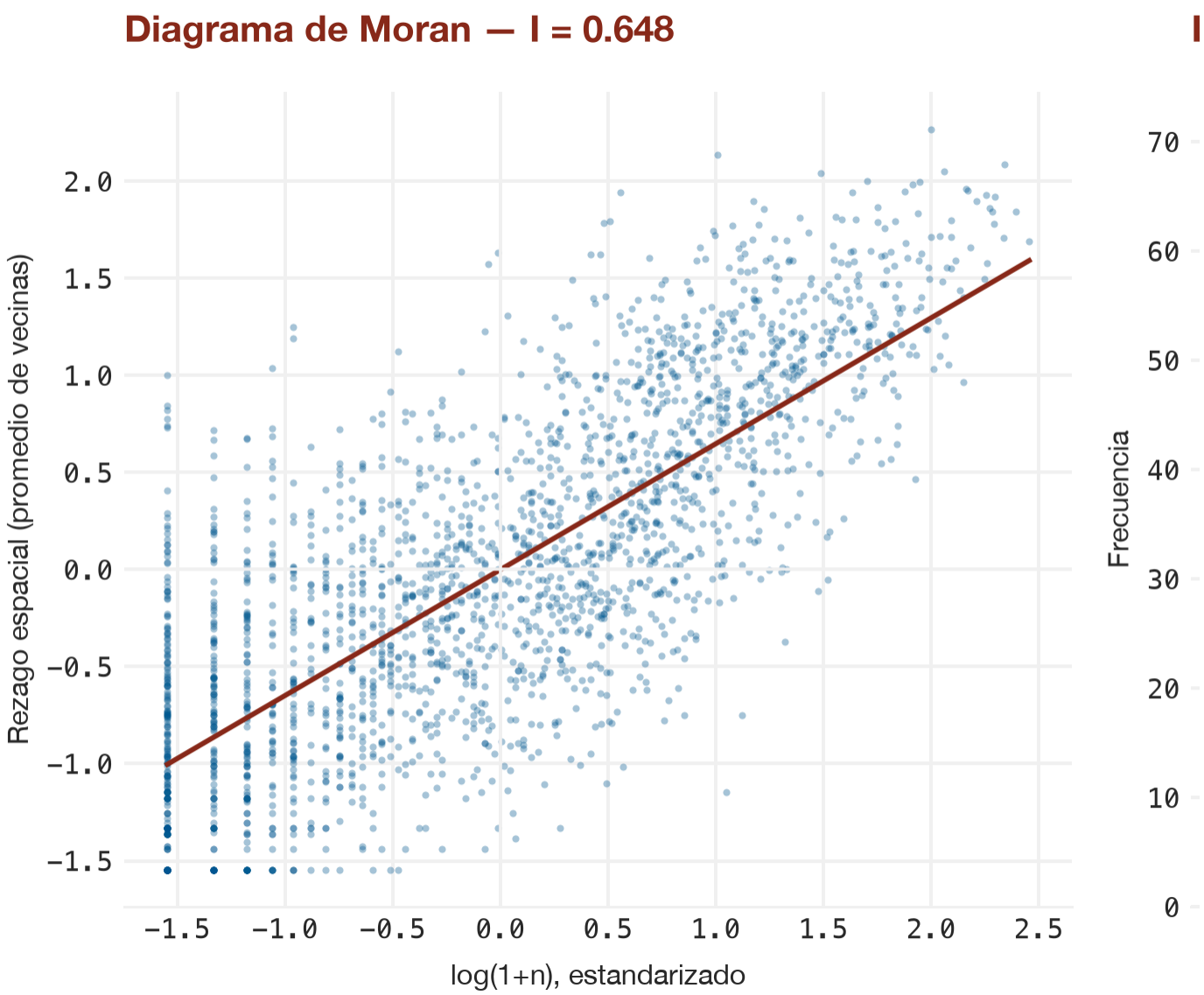
print(f"I de Moran sobre log(1+n): {moran.I:.4f}   (esperado bajo H₀: {moran.EI:.4f})")
print(f"  z de permutación {moran.z_sim:.1f}, p ≤ {moran.p_sim}")
print(f"I de Moran sobre n crudo:  {moran_crudo.I:.4f}   z {moran_crudo.z_sim:.1f}")
```

```
I de Moran sobre log(1+n): 0.6484   (esperado bajo H₀: -0.0004)
  z de permutación 37.0, p ≤ 0.001
I de Moran sobre n crudo:  0.5694   z 33.4
```

```
z = (y - y.mean()) / y.std()
lag = weights.lag_spatial(W, z)

fig, (a1, a2) = plt.subplots(1, 2, figsize=(10, 4.6), gridspec_kw={"width_ratios": [1.15,
a1.scatter(z, lag, s=5, color=AZUL, alpha=0.35, linewidths=0)
lim = np.array([z.min(), z.max()])
a1.plot(lim, moran.I * lim, color=ROJO, lw=1.6)
a1.axhline(0, color=GRIS, lw=1); a1.axvline(0, color=GRIS, lw=1)
a1.set_xlabel("log(1+n), estandarizado")
a1.set_ylabel("Rezago espacial (promedio de vecinas)")
a1.set_title(f"Diagrama de Moran — I = {moran.I:.3f}")
pulir(a1)

a2.hist(moran.sim, bins=40, color=GRIS, edgecolor=BLANCO)
a2.axvline(moran.I, color=ROJO, lw=1.8)
a2.text(moran.I, a2.get_ylim()[1] * 0.92, f"  I obs = {moran.I:.3f}", color=ROJO,
        fontsize=8.5, fontfamily="monospace")
a2.set_xlabel("I bajo permutación aleatoria")
a2.set_ylabel("Frecuencia")
a2.set_title("La H₀ no está ni cerca")
pulir(a2)
plt.tight_layout(); plt.show()
```



Izquierda: diagrama de Moran; cada punto es una celda, la pendiente de la recta es I. Derecha: dónde cae el I observado respecto a 999 permutaciones.

I = 0.65 sobre 2,241 celdas. Para un fenómeno urbano es autocorrelación alta: los accidentes no solo se concentran, se concentran *en zonas contiguas*. El resultado sobre el conteo crudo (0.57) confirma que no es artefacto del logaritmo. La distribución de permutaciones ni se acerca: el valor máximo que produce el azar en 999 barajadas está a más de 30 desviaciones del observado.

Lo que esto dice para el modelo predictivo es directo: **una celda informa sobre sus vecinas.** Un modelo que trate las celdas como independientes desperdicia esa información y, peor, subestima sus errores estándar porque cuenta como independientes observaciones que no lo son.

---

5. Sensibilidad a la escala

El *problema de la unidad de área modificable* (MAUP): cualquier estadístico sobre datos agregados depende de cómo se agregaron. Antes de leer el I = 0.65 como propiedad del fenómeno hay que ver si aguanta al cambiar la rejilla.

```
def construir(lado):
    R = espacial.rejilla_hexagonal(g.total_bounds, lado=lado)
    u = gpd.sjoin(g[["geometry"]], R, predicate="within", how="inner")
    R = R.set_index("hex").join(u.groupby("hex").size().rename("n"), how="inner").reset_i
    w0 = weights.Queen.from_dataframe(R, use_index=False, silence_warnings=True)
    R = R.drop(index=w0.islands).reset_index(drop=True)
    w = weights.Queen.from_dataframe(R, use_index=False, silence_warnings=True)
    w.transform = "r"
    return R, w

filas = []
for lado in (250, 500, 1000):
    R, w = construir(lado)
    m = Moran(np.log1p(R.n.values), w, permutations=199)
    filas.append({"lado (m)": lado, "área (km²)": R.geometry.iloc[0].area / 1e6,
                  "celdas": len(R), "I de Moran": m.I, "z": m.z_sim,
                  "Gini": gini(R.n), "top 5 % concentra": concentracion(R.n)[0.05]})
escalas = pd.DataFrame(filas).set_index("lado (m)")
escalas.style.format({"área (km²)": "{:.2f}", "celdas": "{:,}", "I de Moran": "{:.3f}",
                      "z": "{:.1f}", "Gini": "{:.3f}", "top 5 % concentra": "{:.1%}"})
```

	área (km²)	celdas	I de Moran	z	Gini	top 5 % concentra
lado (m)						
250	0.16	6,145	0.524	52.9	0.746	44.8%
500	0.65	2,241	0.648	37.2	0.756	42.6%
1000	2.60	815	0.721	22.2	0.788	43.5%

Dos lecturas:

- **El I de Moran crece con el tamaño de la celda** (0.52 → 0.65 → 0.72). Eso es el efecto de escala esperado, no una inconsistencia: agregar promedia el ruido local, y lo que queda es la estructura de fondo. Lo importante es que en las tres escalas es alto y a decenas de desviaciones de cero. *La autocorrelación es una propiedad del fenómeno, no de la rejilla.*
- **La concentración no cambia con la escala.** El 5 % superior captura 43-45 % en las tres rejillas. Eso es la firma de una distribución con cola pesada en todos los niveles: los hexágonos calientes de 1 km contienen los hexágonos calientes de 250 m, que contienen las intersecciones calientes.

Los 500 m se conservan como escala de trabajo: suficiente resolución para distinguir corredores viales, suficiente agregación para que la celda típica tenga decenas de eventos y no uno.

---

6. Autocorrelación local: LISA

El I global dice que *hay* estructura; no dice *dónde*. Los indicadores locales de asociación espacial (LISA, Anselin 1995) descomponen el I en una contribución por celda y clasifican cada una según su valor y el de sus vecinas:

Cuadrante	Celda	Vecinas	Lectura
Alto-Alto	alta	altas	Núcleo de un cluster caliente
Bajo-Bajo	baja	bajas	Zona tranquila y homogénea
Alto-Bajo	alta	bajas	Atípico: un punto caliente aislado
Bajo-Alto	baja	altas	Atípico: un hueco en zona caliente

Cada celda tiene su propia prueba por permutaciones. Y aquí entra la advertencia del inicio: 2,241 pruebas al 5 % producen ~112 rechazos por azar. Se aplica **Benjamini-Hochberg**, que controla la proporción esperada de falsos descubrimientos entre los rechazos, y se reporta el antes y el después.

```
lisa = Moran_Local(y, W, permutations=PERM, seed=42)
sig_crudo = lisa.p_sim < 0.05
sig_fdr, _, _, _ = multipletests(lisa.p_sim, alpha=0.05, method="fdr_bh")

CUADRANTES = {1: "Alto-Alto", 2: "Bajo-Alto", 3: "Bajo-Bajo", 4: "Alto-Bajo"}
H["lisa"] = "No significativo"
H.loc[sig_fdr, "lisa"] = pd.Series(lisa.q[sig_fdr]).map(CUADRANTES).values

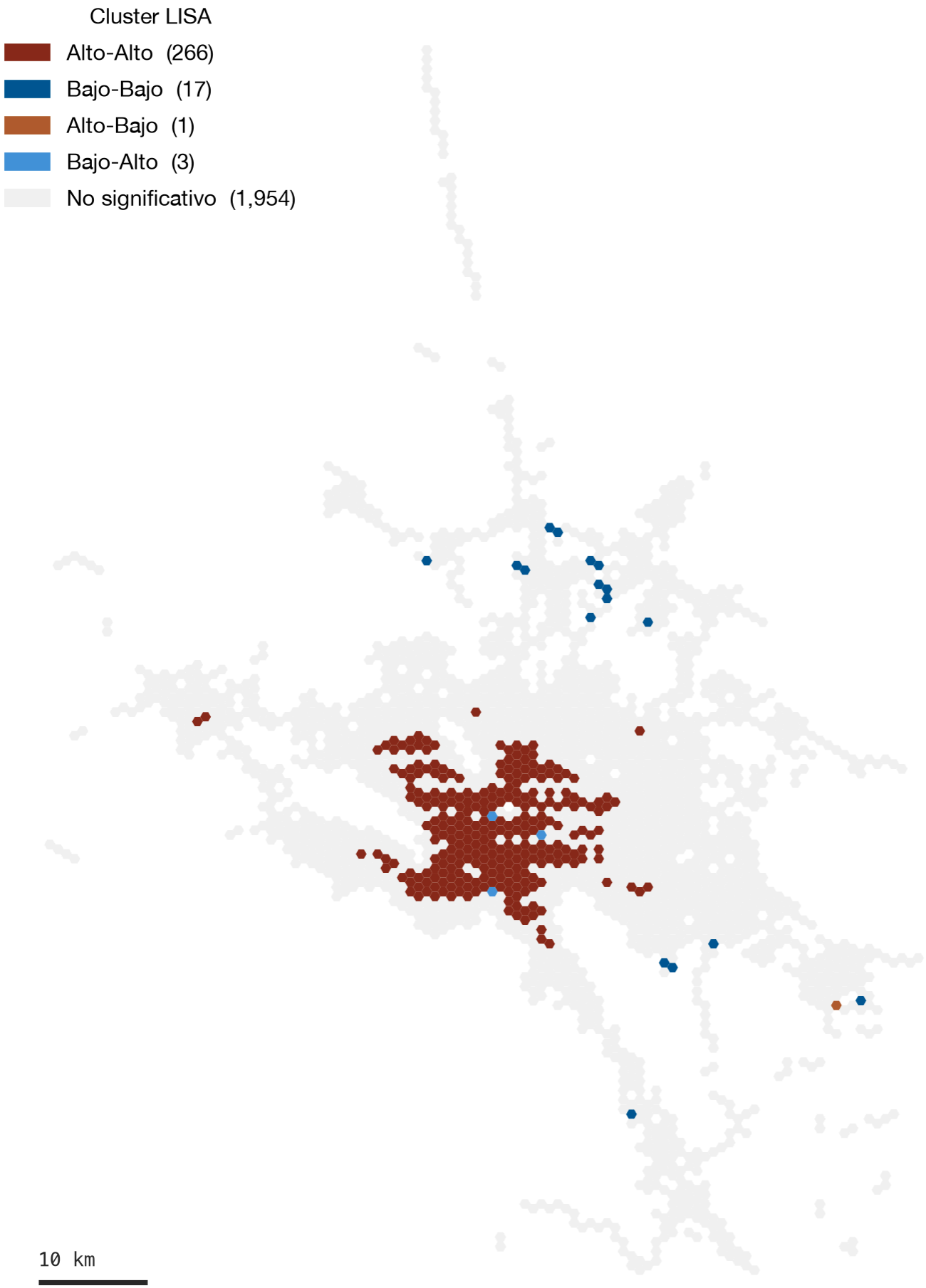
print(f"Celdas significativas al 5 %:  sin corrección {sig_crudo.sum():,}  |  "
      f"con FDR {sig_fdr.sum():,}  |  se caen {sig_crudo.sum() - sig_fdr.sum():,}")
print()
print(H.lisa.value_counts().to_string())
```

```
Celdas significativas al 5 %:  sin corrección 682  |  con FDR 287  |  se caen 395

lisa
No significativo    1954
Alto-Alto            266
Bajo-Bajo             17
Bajo-Alto              3
Alto-Bajo              1
```

```
COLOR_LISA = {"Alto-Alto": ROJO, "Bajo-Bajo": AZUL, "Alto-Bajo": OXIDO,
              "Bajo-Alto": AZUL_RAMPA[2], "No significativo": GRIS}
fig, ax = plt.subplots(figsize=(8.5, 8))
for cat, col in COLOR_LISA.items():
    sub = H[H.lisa == cat]
    if len(sub):
        sub.plot(ax=ax, color=col, linewidth=0)
ax.legend(handles=[Patch(color=c, label=f"{k}  ({(H.lisa == k).sum():,})")
                   for k, c in COLOR_LISA.items()],
          loc="upper left", title="Cluster LISA", title_fontsize=8.5)
mapa(ax, "Clusters de accidentes — LISA")
plt.tight_layout(); plt.show()
```

Clusters de accidentes — LISA



Mapa de clusters LISA sobre log(1+n), 999 permutaciones, FDR al 5 %.

La corrección tira casi el 60 % de los rechazos: **de 682 celdas "significativas" a 287.** Sin ella, el mapa mostraría cuatrocientas celdas de cluster que son ruido. Lo que sobrevive tiene forma reconocible:

- Los **Alto-Alto** (266) son el núcleo urbano continuo: el centro de Monterrey, San Nicolás, Guadalupe y los corredores que los conectan. No son puntos sueltos, es una mancha compacta.
- Los **Bajo-Bajo** son pocos (17), y no porque falten zonas tranquilas: es un efecto de la escala logarítmica. log(1+n) comprime los valores bajos —entre 1 y 10 accidentes hay menos de dos unidades— y estira los altos, así que una celda tranquila rodeada de tranquilas queda *cerca de la media* y su I local es pequeño. El LISA sobre esta variable tiene potencia para detectar lo caliente, no lo frío; el G* de la sección siguiente lo compensa.
- Los atípicos son casi inexistentes (cuatro celdas). Eso significa que **no hay puntos calientes aislados**: cada celda de alto conteo está rodeada de otras altas. El riesgo es de corredor, no de esquina suelta.

---

7. Puntos calientes: Getis-Ord G*

El LISA clasifica por cuadrante; el G* de Getis-Ord responde una pregunta más simple y más operativa: ¿la suma de esta celda *y sus vecinas* es alta respecto al total? Un z positivo grande es un punto caliente; negativo, un punto frío. El asterisco indica que la celda se cuenta a sí misma como vecina.

Misma inferencia por permutación, misma corrección FDR.

```
gistar = G_Local(y, W, transform="r", star=True, permutations=PERM, seed=42)
sig_g_crudo = gistar.p_sim < 0.05
sig_g_fdr, _, _, _ = multipletests(gistar.p_sim, alpha=0.05, method="fdr_bh")

H["gi_z"] = gistar.Zs
H["gi"] = "No significativo"
H.loc[sig_g_fdr & (gistar.Zs > 0), "gi"] = "Caliente"
H.loc[sig_g_fdr & (gistar.Zs < 0), "gi"] = "Frío"

print(f"Celdas significativas al 5 %:  sin corrección {sig_g_crudo.sum():,}  |  "
      f"con FDR {sig_g_fdr.sum():,}")
print()
print(H.gi.value_counts().to_string())
print()
print(f"Accidentes en celdas calientes: {H.loc[H.gi == 'Caliente', 'n'].sum():,} "
      f"({H.loc[H.gi == 'Caliente', 'n'].sum() / H.n.sum():.1%}) en "
      f"{(H.gi == 'Caliente').sum() * AREA_KM2:.0f} km²")
```

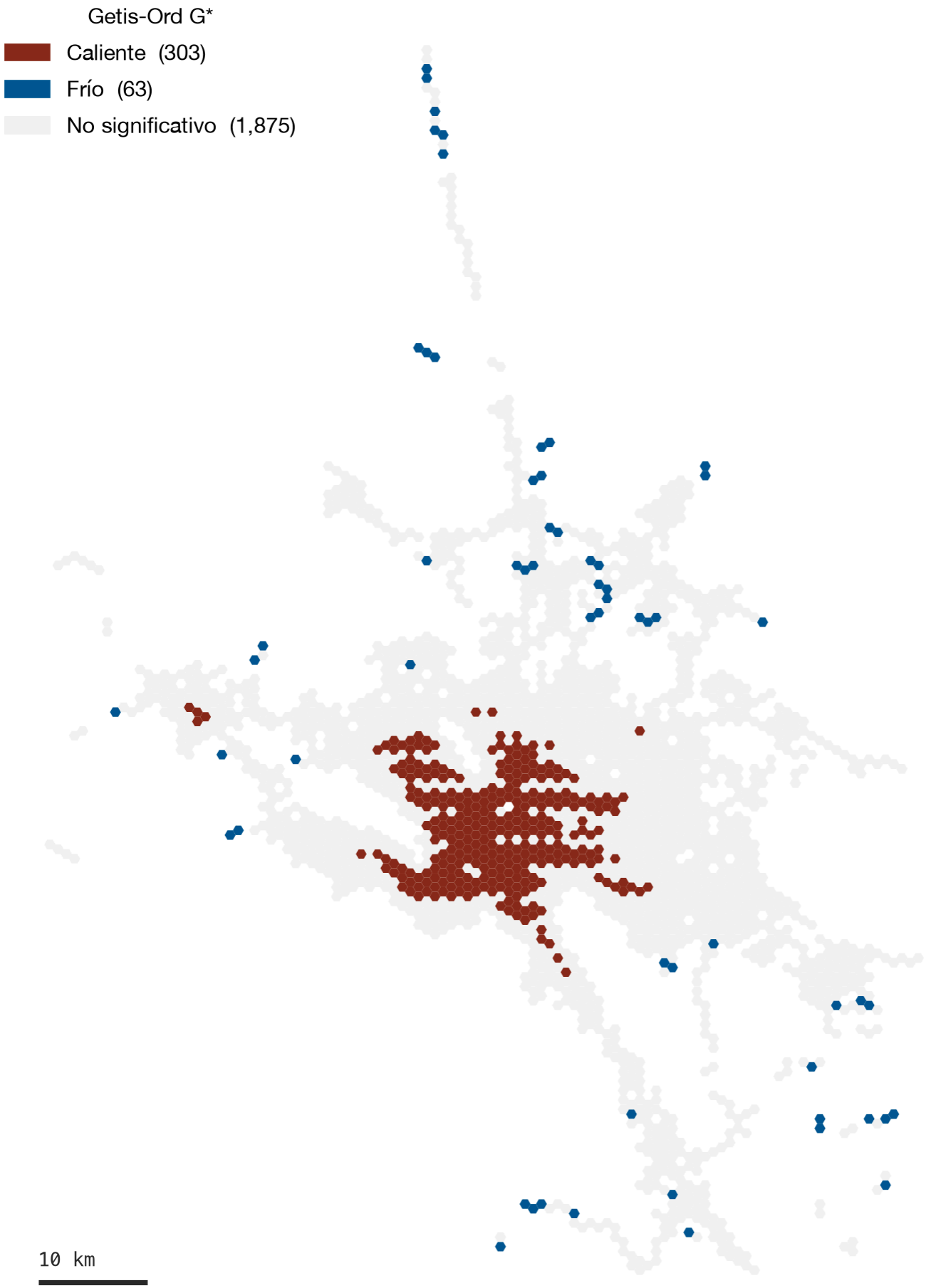
```
Celdas significativas al 5 %:  sin corrección 715  |  con FDR 366
```

```
gi
No significativo    1875
Caliente             303
Frío                  63

Accidentes en celdas calientes: 207,893 (54.8%) en 197 km²
```

```
COLOR_GI = {"Caliente": ROJO, "Frío": AZUL, "No significativo": GRIS}
fig, ax = plt.subplots(figsize=(8.5, 8))
for cat, col in COLOR_GI.items():
    sub = H[H.gi == cat]
    if len(sub):
        sub.plot(ax=ax, color=col, linewidth=0)
ax.legend(handles=[Patch(color=c, label=f"{k}  ({(H.gi == k).sum():,})") for k, c in COLO
          loc="upper left", title="Getis-Ord G*", title_fontsize=8.5)
mapa(ax, "Puntos calientes de accidentes — G*")
plt.tight_layout(); plt.show()
```

Puntos calientes de accidentes — G*



Puntos calientes y fríos de Getis-Ord G*, FDR al 5 %. Solo se colorea lo que sobrevive a la corrección.

303 celdas calientes —el 13.5 % de las que tienen red vial, 197 km²— concentran el 55 % de los accidentes de seis años. El G* detecta más que el LISA (383 contra 287 tras la corrección) porque su pregunta es más laxa: no exige que la celda y sus vecinas sean *parecidas*, solo que la suma del vecindario sea alta. Por eso recupera 80 celdas frías donde el LISA solo sostenía 17.

Los dos estadísticos cuentan la misma historia con distinto detalle y coinciden en lo esencial: un núcleo caliente compacto que sigue la mancha urbana consolidada, y una periferia fría. Que dos estadísticos con distinta lógica converjan es la mejor evidencia de que la estructura es real y no una propiedad del método.

---

8. Persistencia: ¿el mapa de 2019 predice el de 2024?

Esta es la sección que decide si el proyecto predictivo tiene sentido. La autocorrelación espacial dice que hay estructura *en un momento dado*; lo que importa para predecir es si esa estructura **se mantiene** de un año al siguiente. Si el ranking de celdas cambiara cada año, no habría nada que aprender del pasado.

Dos medidas: el I de Moran de cada año por separado (¿la estructura existe todos los años?) y la correlación de Spearman entre los conteos por celda de cada par de años (¿es la *misma* estructura?).

```
moran_anual = pd.Series({a: Moran(np.log1p(H[a].values), W, permutations=0).I for a in AN
spear = pd.DataFrame(index=ANIOS, columns=ANIOS, dtype=float)
for a in ANIOS:
    for b in ANIOS:
        spear.loc[a, b] = stats.spearmanr(H[a], H[b]).statistic

print("I de Moran por año:")
print(moran_anual.round(3).to_string())
print(f"\nSpearman 2019 ↔ 2024: {spear.loc[2019, 2024]:.3f}")
```

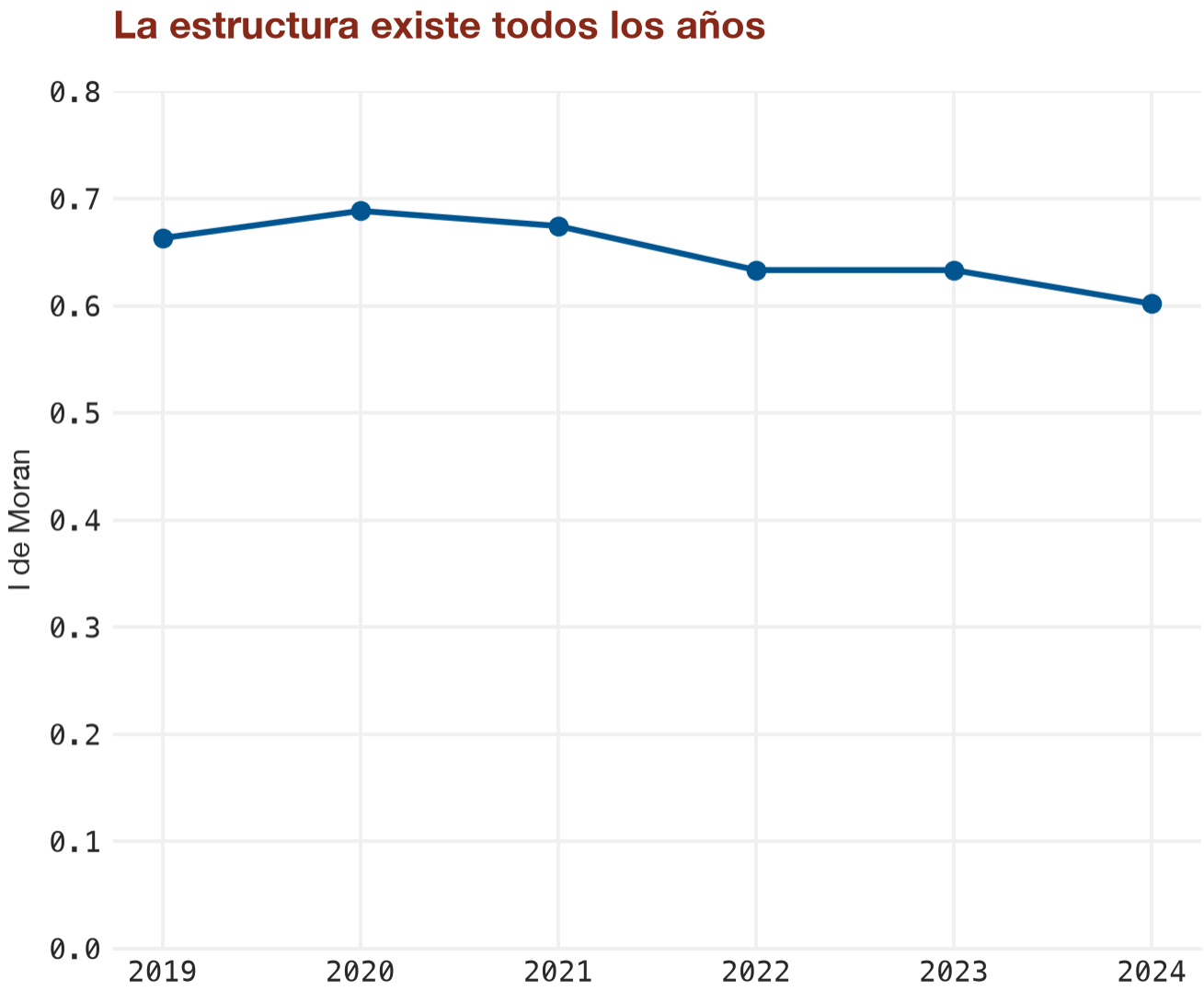
```
I de Moran por año:
2019    0.663
2020    0.689
2021    0.674
2022    0.633
2023    0.633
2024    0.602

Spearman 2019 ↔ 2024: 0.833
```

```
fig, (a1, a2) = plt.subplots(1, 2, figsize=(10.5, 4.4), gridspec_kw={"width_ratios": [1,
a1.plot(moran_anual.index, moran_anual.values, color=AZUL, marker="o", ms=5)
```

```
a1.set_ylim(0, 0.8)
a1.set_ylabel("I de Moran")
a1.set_title("La estructura existe todos los años")
pulir(a1)

im = a2.imshow(spear.values, cmap=SECUENCIAL, vmin=0.7, vmax=1.0)
a2.set_xticks(range(len(ANIOS))); a2.set_xticklabels(ANIOS)
a2.set_yticks(range(len(ANIOS))); a2.set_yticklabels(ANIOS)
for i in range(len(ANIOS)):
    for j in range(len(ANIOS)):
        v = spear.values[i, j]
        a2.text(j, i, f"{v:.2f}", ha="center", va="center", fontsize=8,
                fontfamily="monospace", color=BLANCO if v > 0.86 else GRAFITO)
a2.set_title("…y es la misma estructura")
a2.grid(False)
pulir(a2, ejes=("top", "right", "left", "bottom"))
plt.tight_layout(); plt.show()
```



Izquierda: I de Moran año por año. Derecha: correlación de Spearman entre los conteos por celda de cada par de años.

El ranking de celdas de 2019 correlaciona 0.83 con el de 2024. Cinco años después, con una pandemia en medio, las celdas siguen ordenadas casi igual. La autocorrelación espacial se mantiene entre 0.60 y 0.69 los seis años, sin tendencia.

Hay algo más en la matriz de Spearman que conviene no pasar por alto: **tiene dos bloques.** Dentro de 2019-2021 las correlaciones son 0.87-0.90; dentro de 2022-2024, 0.89-0.91; pero entre un bloque y otro caen a 0.80-0.84. Algo cambió en 2022. El diccionario de datos ofrece la pista: el campo `OID`, que es el identificador que genera QGIS al georreferenciar, **solo existe en los archivos de 2022 a 2024.** La hipótesis más económica es que el INEGI cambió el procedimiento de geocodificación en 2022 y una fracción de los accidentes se asigna a puntos ligeramente distintos desde entonces. No afecta la conclusión —la estructura es persistente en ambos bloques— pero un modelo que entrene con 2019-2023 y valide en 2024 debe saber que la segunda mitad del entrenamiento se parece más a la prueba que la primera.

La prueba operativa: PAI

La correlación de rangos es abstracta. La medida que importa a quien decide es otra: *si uso los años anteriores para elegir dónde intervenir, ¿qué fracción del problema del año siguiente estoy tocando?* Es el **índice de precisión predictiva** (PAI) de la predicción criminal: la fracción de eventos capturada dividida entre la fracción de territorio seleccionada. PAI = 1 es el azar.

```
entren = H[[a for a in ANIOS if a < 2024]].sum(axis=1)
prueba = H[2024]
filas = []
for f in (0.01, 0.05, 0.10, 0.20):
    k = max(1, int(len(H) * f))
    top = entren.nlargest(k).index
    cap = prueba.loc[top].sum() / prueba.sum()
    filas.append({"territorio": f, "celdas": k, "km²": k * AREA_KM2,
                  "accidentes 2024 capturados": cap, "PAI": cap / f})
pai = pd.DataFrame(filas)
(pai.style.hide(axis="index")
    .format({"territorio": "{:.0%}", "celdas": "{:,}", "km²": "{:.0f}",
             "accidentes 2024 capturados": "{:.1%}", "PAI": "{:.1f}"}))
```

territorio	celdas	km²	accidentes 2024 capturados	PAI
1%	22	14	14.4%	14.4
5%	112	73	42.0%	8.4
10%	224	145	58.2%	5.8
20%	448	291	75.1%	3.8

Con el 5 % del territorio elegido a partir de 2019-2023 se captura el 42 % de los accidentes de 2024: PAI 8.4. Con el 10 %, el 58 %. Esto es *sin modelo* —es solo contar y ordenar— y es la cota inferior que cualquier modelo predictivo tiene que superar para justificar su existencia.

9. La escala fina: intersecciones

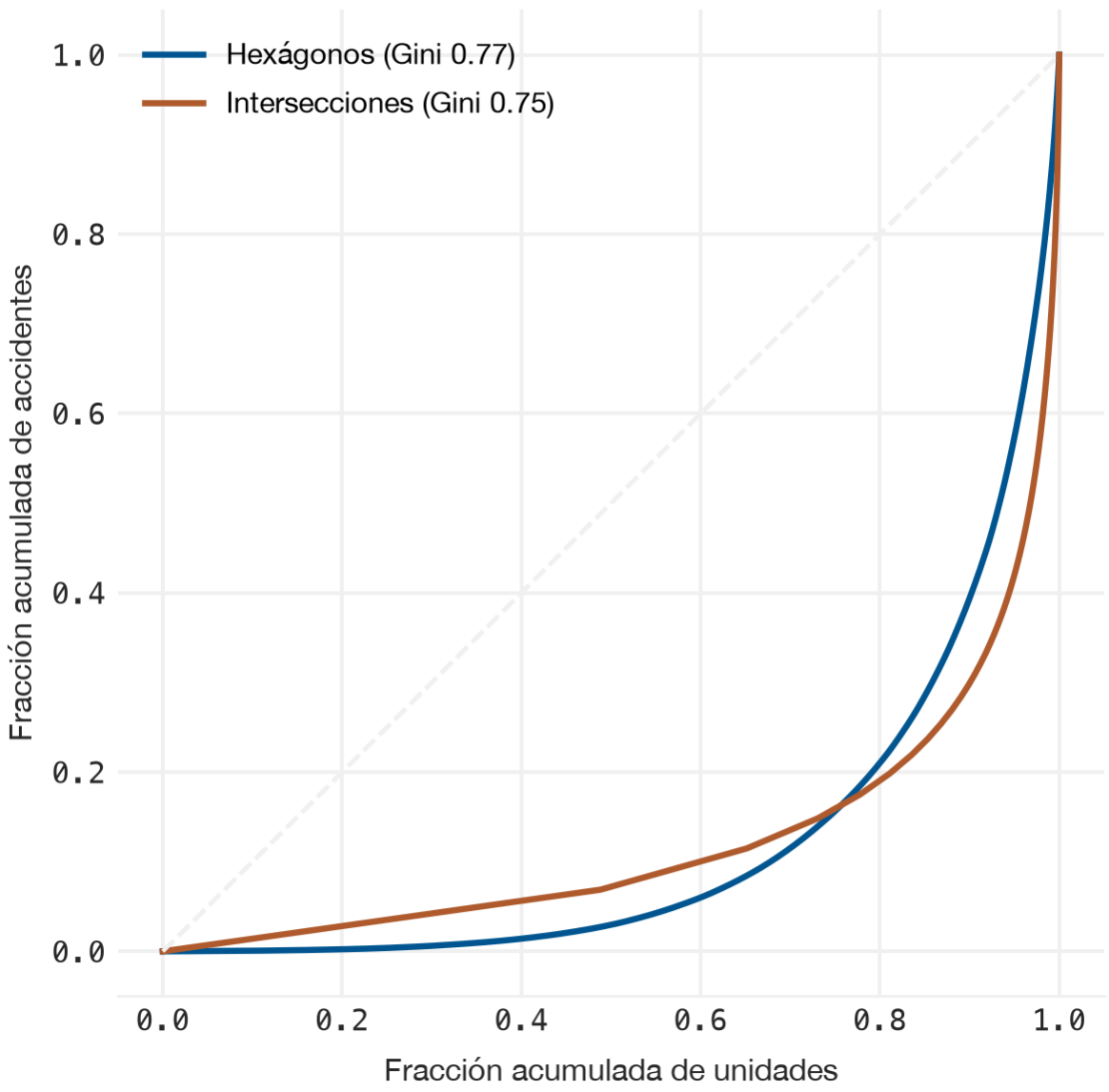
Los hexágonos son la unidad de análisis; las intersecciones son la unidad de intervención. El 96.6 % de los accidentes está marcado como *en intersección* (`URBANA = 1`), y `id_punto_4d` identifica el punto a 10 m de tolerancia. ¿Se sostiene a esa escala lo que se vio arriba?

```
cruces = g[g.en_interseccion]
por_punto = cruces.pivot_table(index="id_punto_4d", columns="ANIO", values="ID",
                               aggfunc="size", fill_value=0)
por_punto["n"] = por_punto[ANIOS].sum(axis=1)
t = por_punto.n.sort_values(ascending=False)

print(f"Accidentes en intersección: {len(cruces):,} ({len(cruces) / N:.1%}) en {len(t):,}
print(f"Puntos con un solo accidente en seis años: {(t == 1).mean():.1%}")
print(f"Gini (puntos): {gini(t):.3f}   |   Gini (hexágonos): {G_HEX:.3f}")
for f, v in concentracion(t).items():
    print(f"  el {f:>4.0%} de los puntos ({int(len(t) * f):>5,}) concentra el {v:.1%}")
```

```
Accidentes en intersección: 366,484 (96.6%) en 51,539 puntos distintos
Puntos con un solo accidente en seis años: 48.7%
Gini (puntos): 0.752   |   Gini (hexágonos): 0.766
  el   1% de los puntos (  515) concentra el 30.5%
  el   5% de los puntos (2,576) concentra el 58.0%
  el  10% de los puntos (5,153) concentra el 70.1%
  el  20% de los puntos (10,307) concentra el 80.9%
```

```
fig, ax = plt.subplots(figsize=(5.2, 5))
for x, etiqueta, col in ((H.n, f"Hexágonos (Gini {G_HEX:.2f})", AZUL),
                         (t, f"Intersecciones (Gini {gini(t):.2f})", OXIDO)):
    px, py = lorenz(x)
    ax.plot(px, py, color=col, label=etiqueta)
ax.plot([0, 1], [0, 1], color=GRIS, lw=1.2, ls="--")
ax.set_xlabel("Fracción acumulada de unidades")
ax.set_ylabel("Fracción acumulada de accidentes")
ax.set_title("La misma concentración en dos escalas")
ax.set_aspect("equal"); ax.legend(loc="upper left")
pulir(ax); plt.tight_layout(); plt.show()
```



Lorenz a dos escalas. La concentración es casi idéntica en hexágonos y en puntos: la cola pesada está en todos los niveles.

```
entren_p = por_punto[[a for a in ANIOS if a < 2024]].sum(axis=1)
prueba_p = por_punto[2024]
k = int(len(por_punto) * 0.05)
top_p = entren_p.nlargest(k).index
cap_p = prueba_p.loc[top_p].sum() / prueba_p.sum()
rho_p = stats.spearmanr(por_punto[2019], por_punto[2024]).statistic

print(f"Spearman 2019 ↔ 2024 sobre puntos: {rho_p:.3f}   (sobre hexágonos: {spear.loc[201
print(f"Top 5 % de puntos según 2019–2023 ({k:,}): captura el {cap_p:.1%} de 2024 → PAI {
```

```
Spearman 2019 ↔ 2024 sobre puntos: 0.131   (sobre hexágonos: 0.833)
Top 5 % de puntos según 2019–2023 (2,576): captura el 49.6% de 2024 → PAI 9.9
```

Aquí aparece la paradoja que define el problema de modelado:

- **La correlación de rangos se desploma**: de 0.83 en hexágonos a 0.13 en puntos. Pero no porque la estructura desaparezca, sino porque la mitad de los puntos tiene un solo accidente en seis años, y ordenar entre "uno" y "cero" es ordenar ruido.
- **La cabeza del ranking, en cambio, es aún más estable**: el top 5 % de puntos captura el 50 % de 2024 (PAI 9.9), mejor que los hexágonos.

Es decir: a escala fina, **las intersecciones que importan son predecibles y las demás son indistinguibles del azar.** Un modelo por intersección tiene que ser capaz de decir "de este punto no sé nada" para los 25,000 puntos con un accidente, y "de este sé mucho" para los 500 con más de cien. Eso es exactamente lo que hace un modelo jerárquico con *shrinkage*, y es la razón por la que el promedio simple por punto no sirve como predictor.

```
# El nombre es el de `docs/cruces_zmm.xlsx` (`nombres_de_cruce`): el par
# homologado más frecuente del punto, no el del primer parte que aparezca.
top15 = (cruces.groupby("id_punto_4d")
         .agg(n=("ID", "size"), municipio=("NOM_MUN", lambda s: s.mode().iloc[0]))
         .sort_values("n", ascending=False).head(15))
top15.insert(0, "cruce", nombres_de_cruce(g).nombre.reindex(top15.index))
top15["por mes"] = top15.n / 72
top15 = top15.reset_index(drop=True)
top15.index = top15.index + 1
top15.style.format({"n": "{:,}", "por mes": "{:.1f}"})
```

	cruce	n	municipio	por mes
1	Fidel Velázquez × Fray Luis de León	1,006	San Nicolás de los Garza	14.0
2	Eugenio Garza Sada × Lázaro Cárdenas	1,004	Monterrey	13.9
3	Acapulco × Miguel Alemán	970	San Nicolás de los Garza	13.5
4	Fidel Velázquez Norte × Manuel L. Barragán	909	San Nicolás de los Garza	12.6
5	Paseo de los Leones × Simón Bolívar	810	Monterrey	11.2
6	Constitución × Félix Uresti Gómez	761	Monterrey	10.6
7	Doctor José Eleuterio González × San Jerónimo	706	Monterrey	9.8
8	Paseo de los Leones × Pedro Infante	698	Monterrey	9.7
9	Calzada del Valle × San Pedro	698	San Pedro Garza García	9.7
10	Chapultepec × Revolución	693	Monterrey	9.6
11	Doctor José Eleuterio González × Terranova	671	Monterrey	9.3
12	Fidel Velázquez × General Bernardo Reyes	670	Monterrey	9.3
13	Constitución × Fundidora	624	Monterrey	8.7
14	Eugenio Garza Sada × Salto del Agua	595	Monterrey	8.3
15	Doctor José Eleuterio González × General Pablo González Garza	593	Monterrey	8.2

Conclusiones

Concentración. Gini espacial de 0.76 a cualquier escala que se mire. El 5 % de las celdas con red vial concentra el 43 % de los accidentes; el 5 % de las intersecciones, el 58 %. El fenómeno es de cola pesada en todos los niveles de agregación.

Autocorrelación. I de Moran de 0.65 sobre hexágonos de 500 m, robusto a la transformación (0.57 sobre conteos crudos) y a la escala (0.52 a 250 m, 0.72 a 1 km). Las celdas no son independientes: cualquier modelo que las trate así subestima sus errores.

Estructura local. LISA y G* coinciden: un núcleo Alto-Alto compacto sobre la mancha urbana consolidada y una periferia fría. Las 303 celdas calientes del G* son el 13.5 % del territorio con red vial y concentran el 55 % de los accidentes. Casi no hay atípicos espaciales (cuatro celdas): el riesgo es de corredor, no de esquina aislada. La corrección FDR retira el 58 % de las celdas LISA y el 48 % de las G* que serían "significativas" sin ella; ese es el tamaño del error que se cometería reportando mapas sin corregir.

Persistencia. El ranking de celdas de 2019 correlaciona 0.83 con el de 2024. Elegir el 5 % del territorio con los cinco años previos captura el 42 % de los accidentes del siguiente (PAI 8.4); a nivel de intersección, el 50 % (PAI 9.9). *Sin modelo.* Esa es la cota que un modelo tiene que superar.

Hallazgo nuevo de este notebook: la matriz de correlación entre años tiene dos bloques, 2019-2021 y 2022-2024, con correlación interna ~0.90 y cruzada ~0.82. Coincide con la aparición del campo `OID` (identificador de QGIS) en 2022. La hipótesis es un cambio en el procedimiento de georreferenciación del INEGI; debería documentarse en el diccionario y considerarse al partir los datos en entrenamiento y prueba.

Lo que este notebook no hace, y por qué

- **Función K de Ripley y densidad kernel.** Sus versiones planas suponen que un accidente puede ocurrir en cualquier punto del plano; aquí solo puede ocurrir sobre la red vial. Las versiones restringidas a red exigen la geometría de las calles (Red Nacional de Caminos u OpenStreetMap), que no está en el repositorio todavía. Correr la versión plana daría un agrupamiento sobreestimado contra un nulo imposible.
- **Covariables.** No hay población, empleo ni kilómetros de vialidad por celda. Este notebook describe *dónde*; explicar *por qué* exige unir con el marco geoestadístico y la red vial, y es el paso siguiente.

Advertencia metodológica

Todos los estadísticos locales se reportan con corrección por múltiples pruebas. Los p de permutación con 999 barajadas no bajan de 0.001, así que un "$p \leq 0.001$" en este documento significa "el observado quedó fuera de las 999 simulaciones", no una probabilidad de un milésimo.